

Capstone Kick-off · Team DTH

Logistics / Delivery Platform RCA

Can an LLM find the root cause of a software failure?

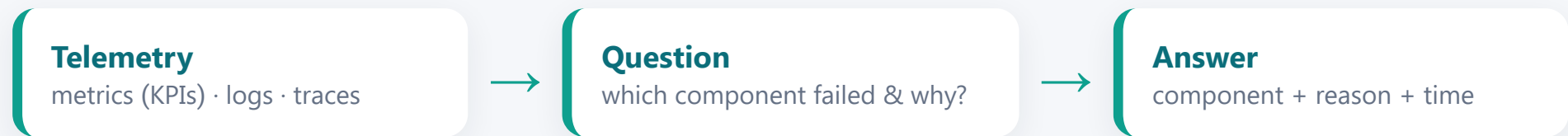
Built on **microsoft/OpenRCA** · OpenRCA system: **Market**

The mission

OpenRCA (ICLR'25) asks software systems: *given a failure and its telemetry, what is the root cause?* It has 335 real failures and 68 GB of metrics, logs and traces.

The opportunity: the best LLM today solves only ~**11%** of cases. That gap is our project — build a working RCA system, add **one novel improvement**, run experiments, and write a short **research paper**.

What is Root Cause Analysis (RCA)?



Example question:

“On 2021-03-25 between 09:00 and 09:30 the delivery-tracking service showed elevated errors. Find the root cause component and reason.”

```
"root cause component": "order-service",  
"root cause reason": "high disk I/O read usage"
```

Our system: Logistics / Delivery Platform RCA

We frame the OpenRCA **Market** system as a **Logistics / Delivery Platform RCA**. Our "services" are:

frontend-gateway

order-service

shipment-service

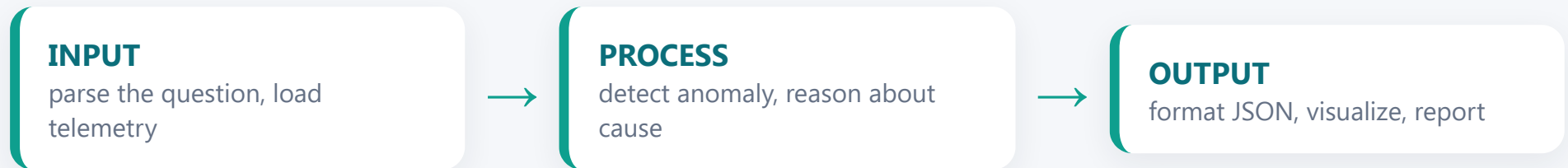
tracking-service

warehouse-service

payment-service

When one of these breaks, our system must point to the right one and explain the reason — automatically, from telemetry.

The pipeline = 3 modules = 3 developers



Each dev owns one module. They connect only through the shared data shapes in `src/schemas.py` — so you can build in parallel.

Meet your module

INPUT

@DuongNg1111

Turn the question into a time window + load the right metrics/logs/traces.

PROCESS

@HoangNguyen2803

Find the anomalous component and explain why (the heart of the project).

OUTPUT

@thanhthanh278

Produce the exact OpenRCA JSON + a chart/report for the demo.

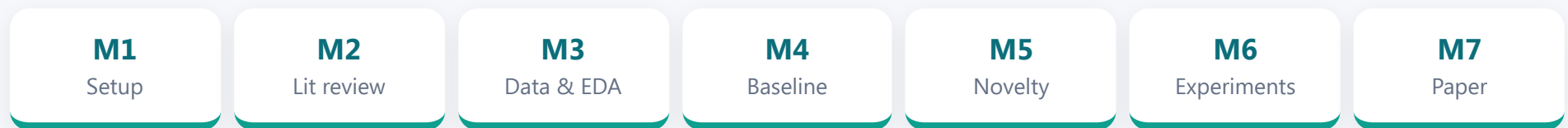
Owners are a suggestion — the leader confirms tonight.

How we work together

- **Branches:** `main` $\leftarrow$ `staging` $\leftarrow$ `dev` $\leftarrow$ `feature/*`. You work on `dev`.
- **main is locked** — you cannot push to it. Open a **Pull Request** (1 approval + green check).
- **Commits:** `feat(process): add z-score detector` · link the Jira issue.
- **Definition of Done:** code + docstring + test/notebook + reviewed PR.

Full cheat-sheet in the handout (Part 5) and `CONTRIBUTING.md`.

The roadmap (Jira project DEV)



Each milestone is an **Epic** in Jira with step-by-step tasks written for newcomers. Each milestone's output becomes a section of the paper.

Tonight's mission 🚀

- 1. Accept the GitHub invite & `git clone` the repo → switch to dev.
- 2. Run `python -m src.pipeline` — see a real prediction appear.
- 3. Open the **Jira board** → filter label `team-dth` → open Epic **M1**.
- 4. Each member **assigns** themselves an M1 task and moves it to **In Progress**.

Step-by-step screenshots/words in the handout, Part 6.

Where the paper (and novelty) comes from

- Heuristic + LLM hybrid that triages cheaply and only calls the LLM on hard cases (cut cost & latency).
- Trace-graph-aware retrieval that walks the service dependency graph to localize the failing service.
- Domain-transfer framing of microservice RCA as a logistics fulfillment pipeline.

We pick one in M5 and prove it beats the baseline in M6.

Let's build it.

Repo: https://github.com/ThienHuynhNgoc/OpenRCA_DTH

Board: <https://tlearning.atlassian.net/jira/software/projects/DEV/list> (filter team=dth)

Handout: [docs/handouts/kickoff_handout.md](#)